



AI Engineer Roadmap 2026

Transform a production-grade full stack engineer (React/Next.js/Node.js/TypeScript/PostgreSQL/Redis/Docker/AWS/REST/Microservices) into a production AI engineer in 6 months (≈ 20 hours/week). [\[1\]](#) [\[2\]](#) [\[3\]](#)

Table of Contents

1. [Introduction](#)
2. [Complete Roadmap Overview](#)
3. [6 Month Timeline](#)
4. [Weekly Learning Plan \(24 Weeks\)](#)
5. [Python Roadmap](#)
6. [Mathematics for Practical AI](#)
7. [Machine Learning Foundations](#)
8. [Deep Learning Foundations](#)
9. [Transformers Deep Dive](#)
10. [LLM Engineering](#)
11. [Retrieval-Augmented Generation \(RAG\)](#)
12. [Vector Databases](#)
13. [AI Agents](#)
14. [Model Context Protocol \(MCP\)](#)
15. [AI Infrastructure](#)
16. [Databases for AI Systems](#)
17. [AI Security](#)
18. [Observability](#)
19. [Production AI Architecture](#)
20. [Portfolio Projects \(20 Projects\)](#)
21. [Open Source Contributions](#)
22. [Interview Preparation](#)
23. [Career Roadmap](#)
24. [Certifications](#)

- 25. [Books](#)
- 26. [Research Papers](#)
- 27. [Best Learning Resources](#)
- 28. [Daily Study Routine](#)
- 29. [Progress Tracking](#)
- 30. [Final Checklist](#)

Introduction

What is AI Engineering?

AI Engineering is the discipline of **designing, building, deploying, and operating AI-powered systems** in production, with a focus on reliability, scalability, security, and business impact rather than just model training. It is closer to software engineering than academic machine learning: you care about APIs, latency, cost, observability, and user experience as much as you care about models. ^[2] ^[4] ^[3] ^[1]

Core responsibilities of an AI Engineer:

- Design and implement LLM/RAG/agent-based features inside products (web, mobile, backend services). ^[1] ^[2]
- Integrate foundation models (OpenAI, Anthropic, Google, open-source) via APIs and SDKs. ^[5] ^[2]
- Build retrieval pipelines (ingestion, chunking, embeddings, ranking) over real-world data sources. ^[4]
- Design agent workflows that call tools, interact with users, and coordinate multi-step tasks. ^[6] ^[7]
- Ensure observability, cost tracking, evaluation, and security across AI features. ^[8] ^[9]

Given your full stack background, you already have strong foundations in APIs, microservices, Docker, and AWS; this roadmap layers AI-specific skills on top. ^[2] ^[1]

AI Engineer vs ML Engineer

- **AI Engineer**
 - Focus: end-to-end AI systems (LLMs, RAG, agents, orchestration, deployment). ^[1] ^[2]
 - Tools: LLM APIs, LangGraph, OpenAI Agents SDK, vector DBs, observability platforms like Langfuse. ^[10] ^[9] ^[6] ^[8]
 - Output: production features (chatbots, copilots, automations) integrated into existing products. ^[2] ^[1]
- **ML Engineer**

- Focus: classic ML models (regression, classification, ranking, recommendation, training pipelines). ^[3]
- Tools: scikit-learn, XGBoost, custom training pipelines, feature stores. ^[3]
- Output: trained models and ML services (e.g., churn prediction, fraud detection). ^[3]

In 2026, many roles blend both, but product-focused positions emphasize AI Engineering skills such as RAG and agents. ^[1] ^[2]

AI Engineer vs Data Scientist

- **Data Scientist**
 - Focus: exploratory analysis, experimentation, statistical modeling, business insights. ^[3]
 - Deliverables: dashboards, reports, experiments, prototypes. ^[3]
- **AI Engineer**
 - Focus: shipping robust AI features with SLAs, scaling to thousands/millions of users. ^[2] ^[1]
 - Deliverables: APIs, microservices, end-user features with CI/CD, monitoring, and incident handling. ^[1] ^[2]

You don't need deep statistical research skills to be an AI Engineer, but you should understand metrics, evaluation, and experimentation enough to avoid flawed deployments. ^[9] ^[4]

AI Engineer vs Research Scientist

- **Research Scientist**
 - Focus: invent new architectures, training algorithms, and foundation models. ^[3]
 - Work: publish papers, design training runs on massive GPU clusters. ^[11]
- **AI Engineer**
 - Focus: use existing models (OpenAI, Anthropic, Mistral, Llama, Mixtral, etc.) and adapt them via prompting, fine-tuning, and RAG. ^[5] ^[2]
 - Work: solve product problems within latency, cost, and reliability constraints. ^[4] ^[2]

Research skills help for frontier roles, but most industry AI Engineer jobs today are application-centric. ^[1] ^[3]

AI Engineer vs Prompt Engineer

- **Prompt Engineer**
 - Focus: crafting and iterating prompts, instructions, and prompt templates. ^[2]
 - Often works inside marketing, content, or data teams.
- **AI Engineer**
 - Uses prompt engineering as one tool among many: system prompts, function/tool calling, orchestration, caching, and evaluation. ^[7] ^[6]

- Owns architecture, integration, reliability, and cost, not just prompts.^[2] ^[1]

Prompt skills are essential but **insufficient**; this roadmap ensures you become a **production AI engineer**, not only a prompt specialist.^[4] ^[2]

Complete Roadmap Overview

High-level skill progression from foundations to production systems.^[1] ^[2]

```
flowchart LR
    A[Python & Ecosystem] --> B[Practical Math]
    B --> C[Classic Machine Learning]
    C --> D[Deep Learning]
    D --> E[Transformers]
    E --> F[LLM Engineering]
    F --> G[RAG Pipelines]
    G --> H[AI Agents]
    H --> I[MCP & Tooling]
    I --> J[Production AI Systems]
    J --> K[Distributed AI & Observability]
```

Conceptual layering:

- You start with Python and practical math to understand what models do under the hood.^[2] ^[1]
- You learn ML and deep learning to reason about evaluation, overfitting, and optimization.^[3]
- You dive into transformers and LLM APIs, then build RAG pipelines, agents, and MCP servers/clients.^[12] ^[13] ^[14] ^[6]
- You finish with infrastructure, observability, security, and distributed architectures for production AI.^[8] ^[9]

6 Month Timeline

Approximate timeline assuming 20 hours/week for 24 weeks.^[1] ^[2]

```
gantt
    title AI Engineer Roadmap (6 Months)
    dateFormat WW
    axisFormat %W

    section Foundations
    Month 1 :done, m1, 01, 4
    Month 2 :active, m2, 05, 4

    section LLM & RAG
    Month 3 :m3, 09, 4
    Month 4 :m4, 13, 4
```

section Agents & Production
Month 5 :m5, 17, 4
Month 6 :m6, 21, 4

Mapping months to themes: ^[2] ^[1]

- **Month 1** – Python, tooling, and practical math.
- **Month 2** – ML fundamentals, evaluation, and PyTorch basics. ^[3]
- **Month 3** – Transformers, LLM APIs, prompt engineering, and basic RAG. ^[4] ^[2]
- **Month 4** – Advanced RAG, vector DBs, evaluation, and observability. ^[9] ^[8]
- **Month 5** – Agents, MCP, orchestrators (LangGraph, Agents SDK, PydanticAI, etc.). ^[15] ^[10] ^[6]
- **Month 6** – Production AI architecture, infra, portfolio, interview prep, and open source. ^[1]
^[2]

Weekly Learning Plan (24 Weeks)

Assumption: 20 hours/week split across learning, practice, and projects. ^[2] ^[1]

Each week includes:

- Objectives
- Topics
- Resources
- Projects / Assignments
- Exercises
- Estimated Hours
- Deliverables

Legend for Hours

- Reading / Videos: 4–6 hours
- Hands-on coding: 10–12 hours
- Projects / Portfolio: 4–6 hours

Week 1 – Python Foundations & Environment

- Objectives
 - Set up a robust Python dev environment.
 - Refresh core syntax and ecosystem basics. ^[1]
- Topics
 - Python basics (syntax, control flow, data structures).

- Virtual environments (venv, uv, poetry).
- Package management (pip, pip-tools). ^[1]
- **Resources**
 - Python Docs – <https://docs.python.org/3/> ^[1]
 - Real Python – <https://realpython.com/> ^[1]
 - “Python Crash Course” – <https://nostarch.com/pythoncrashcourse2e> ^[1]
- **Projects / Assignments**
 - Set up a python-ai-playground mono-repo with apps/, libs/, tests/, and notebooks/ folders. ^[2]
- **Exercises**
 - Implement small scripts: text processing, JSON parsing, CLI tools. ^[1]
- **Estimated Hours**
 - 5h learning, 10h coding, 5h repo setup.
- **Deliverables**
 - Repo with basic utilities (cli.py, utils.py, config.py).

Week 2 – OOP, Typing, and Testing

- **Objectives**
 - Write clean, typed Python code suitable for production. ^[1]
- **Topics**
 - Classes, inheritance, composition.
 - typing and Pydantic models. ^[16] ^[17]
 - Unit testing (pytest), fixtures, mocks. ^[1]
- **Resources**
 - Pydantic Docs – <https://docs.pydantic.dev/> ^[17] ^[16]
 - Pytest Docs – <https://docs.pytest.org/> ^[1]
- **Projects / Assignments**
 - Refactor Week 1 utilities into classes with type hints and Pydantic schemas. ^[16]
- **Exercises**
 - Write data models and validation for an imaginary user/profile service. ^[1]
- **Estimated Hours**
 - 6h learning, 10h coding, 4h tests.
- **Deliverables**
 - Typed models and passing test suite with coverage report.

Week 3 – Asyncio, Generators, Decorators

- **Objectives**
 - Understand async patterns and advanced language features. ^[1]
- **Topics**
 - `async/await`, `asyncio`, concurrency patterns.
 - Generators and iterators.
 - Decorators for logging, retries, caching. ^[1]
- **Resources**
 - AsyncIO Docs – <https://docs.python.org/3/library/asyncio.html> ^[1]
- **Projects / Assignments**
 - Build an async HTTP client wrapper for LLM APIs (OpenAI/Anthropic with dummy endpoints initially). ^[12] ^[6]
- **Exercises**
 - Decorators for retry/backoff and simple cache around HTTP calls.
- **Estimated Hours**
 - 6h learning, 10h coding, 4h experiments.
- **Deliverables**
 - Async client with pluggable middlewares for logging and retries.

Week 4 – FastAPI & API Engineering

- **Objectives**
 - Build production-grade APIs in Python. ^[1]
- **Topics**
 - FastAPI basics (routes, models, dependency injection).
 - Pydantic models for request/response. ^[16]
 - Error handling, logging, middleware. ^[1]
- **Resources**
 - FastAPI Docs – <https://fastapi.tiangolo.com/> ^[1]
 - fastapi GitHub – <https://github.com/tiangolo/fastapi> ^[1]
- **Projects / Assignments**
 - Implement a small FastAPI service with health checks, versioned APIs, and config (12-factor style). ^[1]
- **Exercises**
 - Rate limiting, basic auth, structured logging with `loguru` or `structlog`. ^[1]
- **Estimated Hours**

- 5h learning, 10h coding, 5h tests.
- Deliverables
 - Dockerized FastAPI service ready to host AI endpoints (e.g., /chat, /embed).

Week 5 – Practical Math for ML & LLMs

- Objectives
 - Gain intuition for vectors, matrices, probability, and optimization. ^[3]
- Topics
 - Linear algebra: vectors, matrices, matrix multiplication. ^[3]
 - Calculus: derivatives and gradients.
 - Probability & statistics: distributions, expectation, variance. ^[3]
 - Optimization: gradient descent, learning rates. ^[3]
- Resources
 - “Mathematics for Machine Learning” (book). ^[3]
 - Khan Academy – linear algebra and calculus playlists. ^[3]
- Projects / Assignments
 - Implement gradient descent for simple functions and logistic regression.
 - Simulate cosine similarity between textual embeddings using dummy vectors. ^[2]
- Exercises
 - Hand-compute small matrix multiplications used in attention ($Q \cdot K^T$). ^[11]
- Estimated Hours
 - 8h learning, 8h coding, 4h exercises.
- Deliverables
 - Notebook explaining math behind embeddings and attention with visualizations.

Week 6 – Classical ML with scikit-learn

- Objectives
 - Understand classic ML patterns end-to-end. ^[3]
- Topics
 - Regression, classification, clustering.
 - Train/validation/test splits, cross-validation. ^[3]
 - Metrics: accuracy, precision/recall, F1, ROC-AUC, MSE. ^[3]
 - Overfitting, regularization. ^[3]
- Resources

- Scikit-learn Docs – <https://scikit-learn.org/>^[3]
- “Hands-On ML with Scikit-Learn, Keras & TensorFlow”.^[3]
- **Projects / Assignments**
 - Build a spam vs ham classifier API and expose via FastAPI.^[3]
- **Exercises**
 - Compare models: logistic regression vs random forest vs XGBoost, log metrics.^[3]
- **Estimated Hours**
 - 8h learning, 10h coding, 2h review.
- **Deliverables**
 - ML microservice with metrics logged and simple monitoring.

Week 7 – PyTorch & Neural Networks

- **Objectives**
 - Understand deep learning basics, tensors, and training loops.^[3]
- **Topics**
 - PyTorch tensors and autograd.
 - Simple feedforward networks and nonlinear activations.^[3]
 - Loss functions and optimizers.
- **Resources**
 - PyTorch Docs – <https://pytorch.org/docs/>^[3]
 - Official tutorials – <https://pytorch.org/tutorials/>^[3]
- **Projects / Assignments**
 - Train a small MLP on a tabular dataset (e.g., UCI dataset) with early stopping.^[3]
- **Exercises**
 - Implement custom training loop with logging and evaluation splits.
- **Estimated Hours**
 - 8h learning, 10h coding, 2h experiments.
- **Deliverables**
 - Notebook with training, validation, and loss curves.

Week 8 – CNNs, RNNs, and Intro Transformers

- **Objectives**
 - Get a sense of sequence and image models. ^[3]
- **Topics**
 - CNN basics for images.
 - RNN/LSTM basics for sequences. ^[3]
 - Intro to transformer blocks (encoder layer).
- **Resources**
 - “Deep Learning” (Goodfellow) – CNN/RNN chapters. ^[3]
- **Projects / Assignments**
 - Image classifier with CNN on CIFAR-10 or similar. ^[3]
- **Exercises**
 - Compare RNN vs transformer encoder on a toy text sequence task. ^[2]
- **Estimated Hours**
 - 8h learning, 10h coding, 2h review.
- **Deliverables**
 - Repo with CNN-based classifier and minimal transformer experiment.

Week 9 – Transformers, Attention, and Embeddings

- **Objectives**
 - Understand the internal mechanics of transformers. ^[11]
- **Topics**
 - Self-attention, multi-head attention, positional encodings. ^[11]
 - Tokenization (BPE, SentencePiece). ^[2]
 - Context window, KV cache, RoPE, MoE. ^[2]
- **Resources**
 - “Attention Is All You Need”. ^[11]
 - Hugging Face Transformers Docs – <https://huggingface.co/docs/transformers> ^[18]
- **Projects / Assignments**
 - Use Hugging Face Transformers to run inference with a small model (e.g., `distilbert-base-uncased`). ^[18]
- **Exercises**
 - Visualize attention weights for short sentences. ^[11]
- **Estimated Hours**

- 8h reading/papers, 8h coding, 4h experiments.
- **Deliverables**
 - Notebook explaining transformer internals with diagrams and code.

Week 10 – LLM APIs & Prompt Engineering

- **Objectives**
 - Work with hosted LLM APIs and prompt design. ^[2]
- **Topics**
 - OpenAI, Anthropic, Mistral APIs. ^[5] ^[2]
 - System prompts, few-shot prompting, chain-of-thought. ^[6]
 - JSON/structured output, function/tool calling. ^[7] ^[6]
- **Resources**
 - OpenAI API Docs – <https://platform.openai.com/docs> ^[6] ^[7]
 - Anthropic Docs – <https://docs.anthropic.com/> ^[12]
 - Mistral Docs – <https://docs.mistral.ai/> ^[2]
- **Projects / Assignments**
 - Build a simple chat API using FastAPI + OpenAI/Anthropic with configurable prompts. ^[12] ^[6]
- **Exercises**
 - Design robust system prompts for different personas and log performance.
- **Estimated Hours**
 - 6h docs/videos, 10h coding, 4h experiments.
- **Deliverables**
 - LLM-backed chat microservice with prompt templates and environment-based configs.

Week 11 – RAG Fundamentals

- **Objectives**
 - Implement basic RAG over documents. ^[2] ^[1]
- **Topics**
 - Embeddings, chunking, metadata. ^[4]
 - Vector search vs BM25. ^[4]
- **Resources**
 - Pinecone Docs – <https://docs.pinecone.io/> ^[1]
 - Qdrant Docs – <https://qdrant.tech/documentation/> ^[1]

- LangChain RAG cookbook – https://python.langchain.com/docs/use_cases/question_answerin_g/^[10]
- **Projects / Assignments**
 - Build RAG over a small document set (e.g., your own Next.js/Express docs or API specs).^[4]
- **Exercises**
 - Compare naive fixed-size chunking vs semantic splitting, measure answer quality.^[4]
- **Estimated Hours**
 - 6h learning, 10h coding, 4h evaluation.
- **Deliverables**
 - RAG API with evaluation notebook and configuration toggles.

Week 12 – Advanced RAG & Evaluation

- **Objectives**
 - Add reranking, hybrid search, and evaluation.^{[9] [4]}
- **Topics**
 - Hybrid (BM25 + vectors) search with weighted scores.^[4]
 - Reranking and scoring via LLM or specialized models.^[9]
 - RAG evaluation with LLM-as-a-judge.^[9]
- **Resources**
 - Langfuse Observability & evals – <https://langfuse.com/docs>^{[8] [9]}
 - LangChain RAG evaluation docs.^[10]
- **Projects / Assignments**
 - Implement hybrid RAG with evaluation pipeline that logs quality metrics to Langfuse.^[9]
- **Exercises**
 - Track answer correctness vs retrieval quality on a curated test set.^[9]
- **Estimated Hours**
 - 8h learning, 10h coding, 2h review.
- **Deliverables**
 - RAG service with evaluation metrics and Langfuse dashboards.

Week 13 – Vector Databases in Depth

- Objectives
 - Compare and use multiple vector DBs. ^[1]
- Topics
 - pgvector, Qdrant, Weaviate, Milvus, Pinecone, Chroma, FAISS. ^[1]
- Resources
 - pgvector – <https://github.com/pgvector/pgvector> ^[1]
 - Qdrant – <https://github.com/qdrant/qdrant> ^[1]
 - Weaviate – <https://weaviate.io/developers> ^[1]
 - Pinecone – <https://www.pinecone.io/docs/> ^[1]
 - FAISS – <https://github.com/facebookresearch/faiss> ^[1]
- Projects / Assignments
 - Implement abstractions for different vector DB backends and feature flags to switch. ^[1]
- Exercises
 - Benchmark latency and recall on a sample dataset across DBs. ^[1]
- Estimated Hours
 - 8h learning, 10h coding, 2h experiments.
- Deliverables
 - Vector DB comparison notebook and API wrapper library.

Week 14 – Agents Basics (Single-Agent Workflows)

- Objectives
 - Learn single-agent orchestration patterns. ^[7] ^[6]
- Topics
 - Tool calling, planning, memory.
 - Single agent with tools (e.g., search, calculator, RAG). ^[6] ^[7]
- Resources
 - OpenAI Agents SDK – <https://developers.openai.com/api/docs/guides/agents> ^[6]
 - Agents SDK Guide – <https://platform.openai.com/docs/guides/agents-sdk> ^[7]
- Projects / Assignments
 - Build a single agent that calls tools for web search, calculations, and RAG over your docs. ^[6]
- Exercises
 - Add memory and basic state; log sessions and outcomes. ^[7]

- **Estimated Hours**
 - 8h learning, 10h coding, 2h evaluation.
- **Deliverables**
 - Agent microservice using Agents SDK with tracing enabled.

Week 15 – Multi-Agent Systems & LangGraph

- **Objectives**
 - Design multi-agent workflows and graphs. ^[15] ^[10]
- **Topics**
 - LangGraph nodes, edges, state management.
 - CrewAI, PydanticAI, SmolAgents comparison. ^[19] ^[15]
- **Resources**
 - LangGraph GitHub – <https://github.com/langchain-ai/langgraph> ^[10]
 - LangGraph 101 – <https://github.com/langchain-ai/langgraph-101> ^[20]
 - Multi-agent comparison – <https://medium.com/@mpuig/which-multi-agent-framework-should-run-your-enterprise-ai-abdc8e09ad89> ^[15]
 - Field-tested comparison – <https://ianas.fr/en/blog/2026/06/02/crewai-langchain-langgraph-comparatif-pragmatique/> ^[19]
- **Projects / Assignments**
 - Build an agent graph: planner → RAG agent → actions agent with LangGraph. ^[10]
- **Exercises**
 - Add human-in-the-loop steps and error branches. ^[10]
- **Estimated Hours**
 - 8h learning, 10h coding, 2h testing.
- **Deliverables**
 - LangGraph-based agent workflow, Dockerized and documented.

Week 16 – MCP (Model Context Protocol)

- **Objectives**
 - Learn MCP concepts and integrate servers/clients. ^[13] ^[14] ^[12]
- **Topics**
 - MCP spec, servers, clients, tools, transport (stdio, HTTP + SSE). ^[14] ^[13]
 - Authentication and security patterns. ^[12]
- **Resources**

- Anthropic MCP announcement – <https://www.anthropic.com/news/model-context-protocol> ^[12]
- MCP Wikipedia and site – <https://modelcontextprotocol.io> ^[13] ^[14]
- OpenAI adoption of MCP – <https://getathenic.com/blog/openai-model-context-protocol-announcement> ^[21]
- **Projects / Assignments**
 - Build an MCP server exposing PostgreSQL and GitHub data with JSON-RPC. ^[13] ^[12]
- **Exercises**
 - Connect Claude Desktop / Agents SDK client to your server and test flows. ^[12] ^[6]
- **Estimated Hours**
 - 8h learning, 10h coding, 2h security review.
- **Deliverables**
 - MCP server repo with docs and example client integration.

Week 17 – AI Infrastructure (Docker, K8s, GPUs)

- **Objectives**
 - Prepare infra for serving AI systems. ^[2] ^[1]
- **Topics**
 - Docker best practices for AI services (slim images, healthchecks). ^[2]
 - Kubernetes basics, Helm charts for AI workloads. ^[1]
 - GPU scheduling, autoscaling, node pools. ^[2]
- **Resources**
 - Kubernetes Docs – <https://kubernetes.io/docs/> ^[1]
 - LangGraph deploy practices – <https://docs.langchain.com/oss/python/langgraph> ^[10]
- **Projects / Assignments**
 - Deploy RAG and agent services into a K8s cluster with Ingress and secrets. ^[1]
- **Exercises**
 - Measure latency and scale under load using k6 or similar tools. ^[1]
- **Estimated Hours**
 - 8h infra learning, 10h deployments, 2h monitoring.
- **Deliverables**
 - K8s manifests or Helm chart for your AI stack.

Week 18 – Inference Servers (vLLM, TensorRT-LLM, Ray Serve, KServe)

- **Objectives**
 - Serve open-source models efficiently. ^[11] ^[2]
- **Topics**
 - vLLM architecture and OpenAI-compatible endpoints. ^[2]
 - TensorRT-LLM for NVIDIA GPUs. ^[11]
 - Ray Serve and KServe for distributed inference. ^[2]
- **Resources**
 - vLLM GitHub – <https://github.com/vllm-project/vllm> ^[2]
 - NVIDIA TensorRT-LLM – <https://developer.nvidia.com/tensorrt-llm> ^[11]
- **Projects / Assignments**
 - Deploy a local Llama/Mistral model using vLLM behind an API gateway. ^[5] ^[2]
- **Exercises**
 - Compare local vs hosted LLM latency/cost for specific endpoints. ^[2]
- **Estimated Hours**
 - 8h learning, 10h coding, 2h benchmarks.
- **Deliverables**
 - Inference server with health checks, metrics, and autoscaling policy.

Week 19 – Observability & Evaluation (Langfuse, LangSmith, Phoenix)

- **Objectives**
 - Instrument AI stack for tracing, evaluation, and metrics. ^[8] ^[9]
- **Topics**
 - Langfuse traces, spans, sessions, cost tracking. ^[8] ^[9]
 - LangSmith for agent graphs and debug. ^[10] ^[9]
 - Phoenix-style local-first observability for LLM tracing. ^[22]
- **Resources**
 - Langfuse Docs – <https://langfuse.com/docs> ^[9]
 - Langfuse observability overview – <https://langfuse.com/docs/observability/overview> ^[8]
 - Phoenix observability blog – <https://michaeljohnpena.com/blog/2024-10-18-phoenix-observability/> ^[22]
- **Projects / Assignments**
 - Instrument all AI services with tracing and evaluation in Langfuse. ^[9]
- **Exercises**

- Create dashboards for latency, cost, and quality across features. ^[9]
- **Estimated Hours**
 - 8h learning, 10h instrumentation, 2h dashboards.
- **Deliverables**
 - Observability stack integrated with your AI platform (traces + evals).

Week 20 – AI Security & Governance

- **Objectives**
 - Design secure, compliant AI features. ^[4]
- **Topics**
 - Prompt injection and jailbreak defenses. ^[4]
 - PII handling, secrets management, RBAC. ^[4]
 - Supply chain and model security (dependency and model origin). ^[4]
- **Resources**
 - OWASP LLM Top 10 – <https://owasp.org/www-project-top-10-for-large-language-model-applications/> ^[4]
- **Projects / Assignments**
 - Add guardrails and filters to your agents (content filters, policy checks). ^[7] ^[6]
- **Exercises**
 - Run red-team tests against your RAG and agents for injection attempts. ^[4]
- **Estimated Hours**
 - 8h security reading, 10h implementation, 2h review.
- **Deliverables**
 - Security checklist and implemented guardrails.

Week 21 – Production AI Architecture & System Design

- **Objectives**
 - Design end-to-end AI platforms. ^[4] ^[2]
- **Topics**
 - API gateway, LLM router, workers, queues. ^[2]
 - Redis, databases, vector DBs, caching. ^[4] ^[1]
- **Resources**
 - Practical AI engineer architecture posts – LinkedIn and blogs on 2026 AI platforms. ^[8]
^[4]

- **Projects / Assignments**
 - Design a full architecture diagram for a multi-LLM, multi-feature AI platform.
- **Exercises**
 - Prepare system design answers for “Design a RAG system”, “Design an AI copilot”. ^[2] ^[1]
- **Estimated Hours**
 - 8h design learning, 10h diagrams, 2h documentation.
- **Deliverables**
 - Architecture docs and diagrams in Mermaid and Markdown.

Week 22 – Portfolio Projects (Execution)

- **Objectives**
 - Finish 3–5 flagship portfolio projects. ^[2] ^[1]
- **Topics**
 - Polish, robustness, documentation, deployment.
- **Resources**
 - GitHub portfolios from AI engineers and open-source frameworks (LangGraph, vLLM, etc.). ^[23] ^[5] ^[10]
- **Projects / Assignments**
 - Finalize top projects with demos and READMEs.
- **Exercises**
 - Write technical blog posts for each key project. ^[5]
- **Estimated Hours**
 - 4h planning, 12h building, 4h docs.
- **Deliverables**
 - Published repos and live demos at sensible domains (e.g., `ai-copilot.yourdomain.com`).

Week 23 – Interview Prep & Mock Rounds

- **Objectives**
 - Prepare for AI engineer interviews. ^[3]
- **Topics**
 - System design, coding, Python, SQL, AI-specific rounds (LLMs, RAG, agents, security). ^[2] ^[1]
- **Resources**
 - AI engineer interview guides from recent roadmaps. ^[2] ^[3] ^[1]

- **Projects / Assignments**
 - 2–3 full mock interviews (peer or recorded), focusing on AI system design.
- **Exercises**
 - Practice 100 interview questions (see outline below).
- **Estimated Hours**
 - 6h learning, 8h practice, 6h mocks.
- **Deliverables**
 - Interview notes, question bank, and refined narrative of your journey.

Week 24 – Applications, Networking, and Open Source

- **Objectives**
 - Ship, share, and start applying. ^[5] ^[3]
- **Topics**
 - Job applications, networking, open source contributions.
- **Resources**
 - Open-source projects list (LangChain, LangGraph, LlamaIndex, etc.). ^[23] ^[10] ^[11]
- **Projects / Assignments**
 - Submit PRs, write posts, and apply to AI roles with targeted portfolios. ^[5]
- **Exercises**
 - Refine resume and LinkedIn with AI Engineer focus. ^[4]
- **Estimated Hours**
 - 4h resume, 8h PRs, 8h applications.
- **Deliverables**
 - Updated CV, active GitHub profile, open-source contributions, and applied roles.

Python Roadmap

You already know Node.js and TypeScript; Python becomes your AI-centric backend language. ^[2] ^[1]

Core Topics

- Python syntax and idioms (PEP8, context managers). ^[1]
- OOP, dataclasses, and Pydantic models. ^[17] ^[16]
- Type hints (typing), static analysis (mypy, ruff). ^[1]
- Generators, iterators, decorators for cross-cutting concerns.
- asyncio and async patterns for IO-heavy AI workloads. ^[1]

- Packaging (uv, poetry, setuptools).^[10]
- Virtual environments and dependency management.^[1]

FastAPI & Pydantic

FastAPI and Pydantic are the default stack for many production AI services.^[1]

- FastAPI for REST/JSON APIs with async support.
- Pydantic for request/response schemas and validation.^[16]
- JWT auth, rate limiting, error handling middleware.

Testing

- pytest for unit/integration tests.^[1]
- httpx for API tests.
- Coverage and CI integration (GitHub Actions).^[10]

Recommended Resources

Category	Resource	Notes
Docs	Python Docs	Official, comprehensive. ^[1]
Docs	FastAPI Docs	Production API patterns. ^[1]
Docs	Pydantic Docs	Typed models and validation. ^{[16] [17]}
Course	FastAPI Crash Course (YouTube, multiple channels)	Free introduction. ^[1]
GitHub	tiangolo/fastapi	Reference implementation. ^[1]
GitHub	pydantic/pydantic-ai	Agent framework using Pydantic. ^{[16] [24]}

Checklist:

- [] Comfortable writing typed Python services with FastAPI.
- [] Use Pydantic models for all external boundaries.
- [] Package and ship via Docker with reproducible builds.

Mathematics for Practical AI

Only the math needed to reason about models and avoid mistakes.^[3]

Linear Algebra

- Vectors and norms – used for embeddings and similarity.^[3]
- Matrices and matrix multiplication – core of attention and neural network layers.^[11]

Why it matters:

- Embedding operations (dot products, cosine similarity).
- Attention calculations over sequences ($Q \cdot K^T$ scaled, softmax). ^[11]

Calculus & Optimization

- Derivatives and gradients.
- Gradient descent and variants (Adam, RMSProp). ^[3]

Why it matters:

- Training loops in ML/DL.
- Understanding how models learn and converge; tuning learning rates and regularization. ^[3]

Probability & Statistics

- Basic distributions, expectation, variance. ^[3]
- Conditional probability and Bayes intuition.
- Confidence and uncertainty in predictions.

Why it matters:

- Evaluation metrics and error analysis.
- A/B testing of AI features and user behavior changes. ^[3]

Information Theory & Similarity

- Entropy and cross-entropy. ^[3]
- Cosine similarity for embeddings. ^[2]

Why it matters:

- Loss functions for classification.
- Embedding similarity and retrieval scoring.

Attention Mathematics

At a high level, a single attention head performs: ^[11]

- Compute queries, keys, values (linear projections).
- Score = dot product between queries and keys.
- Normalize scores with softmax.
- Weighted sum of values to produce outputs.

You need to understand **conceptually** how tokens attend to each other, not derive every formula in detail. ^[11] ^[2]

Machine Learning Foundations

Goal: treat ML as a system, not only a subject. ^[4] ^[3]

Core Concepts

- Supervised vs unsupervised learning. ^[3]
- Regression (linear, logistic). ^[3]
- Classification (binary, multi-class).
- Clustering (k-means, hierarchical). ^[3]

Model Evaluation

- Train/validation/test splits. ^[3]
- Cross-validation.
- Metrics: MSE, MAE, accuracy, precision/recall, F1, ROC-AUC. ^[3]

Why it matters:

- Many AI failures come from wrong evaluation, not bad models (e.g., wrong metric, skewed data). ^[4]

Overfitting & Regularization

- Overfitting, bias/variance tradeoff. ^[3]
- L1/L2 regularization and penalties.
- Early stopping and model selection.

Algorithms

- Random Forest for robust non-linear modeling. ^[3]
- XGBoost for strong tabular performance. ^[3]
- Logistic regression, linear regression, SVMs.

Tools

- scikit-learn for pipelines and standard algorithms. ^[3]
- XGBoost library for gradient boosting. ^[3]

Projects

- Churn prediction API for a SaaS dataset. ^[3]
- Spam classifier integrated with a Next.js frontend. ^[3]

Deep Learning Foundations

Deep learning is required for understanding transformers and advanced tasks, even if you don't train giant models. ^[3]

Core Topics

- Neural networks (feedforward, activation functions like ReLU, GELU).
- Backpropagation and gradients. ^[3]
- GPU acceleration and CUDA concepts (tensors on GPU, memory).

Frameworks

- PyTorch – preferred for research-like work and many OSS models. ^[3]
- TensorFlow/Keras – widely used, especially in some enterprise stacks. ^[3]

Architectures

- CNNs for images (conv, pooling, feature maps). ^[3]
- RNNs/LSTMs for sequences (time steps, hidden states). ^[3]
- Basic transformer encoder/decoder (multi-head attention, MLP blocks). ^[11]

Training & Inference

- Training loops, data loaders, batching. ^[3]
- Inference optimization (batching, quantization, caching). ^[2]

Projects

- Image classifier for product images or logos.
- Text classifier for customer feedback categories.

Transformers Deep Dive

Transformers are the foundation of modern LLMs. ^[11]

Key Concepts

- Encoder vs decoder vs encoder-decoder (e.g., BERT vs GPT vs T5). ^[18] ^[2]
- Self-attention and cross-attention. ^[11]
- Multi-head attention combining different representation subspaces. ^[11]

Embeddings & Positional Encoding

- Token embeddings represent discrete tokens in vector space. ^[18] ^[2]
- Positional encodings (sinusoidal, learned) provide order information. ^[11]

Tokenization

- BPE (Byte-Pair Encoding) used by many models. ^[2]
- SentencePiece for subword tokenization in multilingual contexts. ^[2]

Inference Concepts

- Context window and truncation; sliding windows for long texts. ^[2]
- Temperature, top-p, top-k sampling for diversity control. ^[2]
- KV cache to reuse key/value pairs across tokens. ^[2]
- RoPE (Rotary Embeddings) for better extrapolation in attention. ^[2]
- Mixture of Experts (MoE) for conditional computation. ^[2]

Quantization

- 8-bit/4-bit quantization for efficient inference on limited hardware. ^[2]

Vision Transformers

- ViT for images, patch-based embeddings fed to transformer encoder. ^[2]

LLM Engineering

Focus: building LLM-backed applications that behave reliably in production. ^[2] ^[1]

Prompt Engineering

- System prompts for global behavior.
- User prompts for instructions and queries.
- Tool prompts to describe functions and constraints. ^[6] ^[7]
- Prompt templates and versioning in your code or tools like Langfuse. ^[9]

Structured Output & Tool Calling

- JSON mode and schema validation via LLM or Pydantic. ^[7] ^[6]
- Function calling for tools (maps to Python functions, HTTP endpoints, MCP tools). ^[6] ^[7]

Streaming & Context Engineering

- Streaming responses via SSE/WebSockets for UX. ^[6]
- Context window management and chunking for long inputs (history summarization, truncation). ^[2]

Guardrails & Safety

- Input validation (schema, policy).
- Output post-processing, redaction, and policy enforcement (e.g., PII removal). ^[4]
- Guardrails and human review integration in Agents SDK and LangGraph. ^{[7] [10]}

Memory & Model Routing

- Short-term memory (session context).
- Long-term memory via vector DBs or relational DBs. ^[4]
- Routing across multiple models (cheap vs capable) based on task type and budget. ^[2]

Cost Optimization

- Track tokens and cost per feature via Langfuse or internal logs. ^{[8] [9]}
- Use smaller models for simple tasks, reserved big models for complex ones. ^[2]

Projects

- Multi-LLM router service (OpenAI + local vLLM).
- Prompt management and evaluation dashboard integrated with Langfuse. ^[9]

Retrieval-Augmented Generation (RAG)

RAG is a primary job market driver in 2026. ^{[4] [2] [1]}

Core Concepts

- Embeddings: numeric representations of text via models like text-embedding-3-large or all-MiniLM-L6-v2. ^{[18] [2]}
- Chunking strategies: by paragraphs, sections, or semantic units (e.g., headings). ^[4]
- Metadata: document source, timestamps, tags, user/org IDs. ^[4]

Search

- Pure vector search (kNN over embeddings). ^[1]
- BM25 keyword search via Elasticsearch or similar. ^[4]
- Hybrid search (combine keyword and vector scoring). ^[4]

Reranking

- Let a model rerank top-k retrieved documents based on context and query. ^[9]

Evaluation

- Answer correctness, source coverage, factuality.
- LLM-as-a-judge evaluation pipelines. ^[9]

Projects

- Documentation assistant for your own APIs (Express/Next.js stack).
- E-commerce product Q&A system grounded in product catalog and reviews.

Vector Databases

Vector DBs are essential for RAG and semantic search. ^[1]

Comparison Table

DB	Type	Hosting	Pros	Cons
pgvector	Postgres extension	Self-hosted	Reuse existing Postgres, strong consistency. ^[1]	Not specialized for massive vector workloads.
Qdrant	Native vector DB	OSS + Cloud	Good performance, hybrid search, open source. ^[1]	Extra infra to manage.
Weaviate	Vector DB + KG	OSS + Cloud	Strong semantic + hybrid capabilities. ^[1]	More complex to operate.
Milvus	Vector DB	OSS	Scales to billions of vectors. ^[1]	Operationally heavier.
Pinecone	Managed vector DB	SaaS	Easy setup, good performance. ^[1]	Vendor lock-in and cost.
Chroma	Embedded/OSS	Local	Simple to start, good for prototypes. ^[1]	Not ideal for large multi-tenant use.
FAISS	Library	Self-managed	Great for custom indexing and research. ^[1]	Needs custom infra and integration.

Checklist:

- [] Use pgvector when you want minimal additional infra and reuse Postgres.
- [] Use Qdrant/Weaviate/Milvus for dedicated, self-hosted vector services.
- [] Use Pinecone for quick managed deployments at early stages.

AI Agents

Agents automate multi-step tasks with tools and memory. [\[25\]](#) [\[15\]](#) [\[6\]](#)

Core Topics

- Planning: break tasks into steps and choose tools. [\[7\]](#) [\[6\]](#)
- Reflection: evaluate outputs and adjust plans.
- Tool calling: interact with APIs, DBs, and systems.
- Memory: maintain state over time (short-term and long-term). [\[10\]](#)
- Workflows and state machines: explicit graphs (LangGraph). [\[10\]](#)
- Human approval: add review and approval steps. [\[7\]](#) [\[10\]](#)
- Long-running tasks and retries (queues, worker orchestration).
- Multi-agent systems (specialist agents that hand off work). [\[25\]](#) [\[15\]](#)

Framework Comparison

Framework	Style	Strengths	When to use
LangGraph	Graph/state machine	Fine-grained control, durable, production-ready. [10] [25]	Complex workflows, multi-step agents.
PydanticAI	Typed agent framework	Strong schema and type safety, Pythonic. [16] [15]	Apps with strict data contracts.
OpenAI Agents SDK	Lightweight SDK	Tight integration with OpenAI tools, MCP, tracing. [6] [7]	If most logic uses OpenAI models/tools.
CrewAI	Team-of-agents abstraction	High-level multi-agent orchestration. [15] [19]	Rapid multi-agent prototyping.
AutoGen	Event-driven chat agents	No-code studio, cross-language support. [15]	Multi-agent chat workflows, enterprise.
SmolAgents	Minimalist, code-writing	Radical flexibility, small codebase. [15]	Research and experimental setups.

Projects

- AI support agent that triages tickets, retrieves customer history, and drafts responses.
- Workflow: planner → data retriever (RAG) → summarizer → email composer → human approval.

Model Context Protocol (MCP)

MCP is an open standard for connecting AI assistants to data sources and tools. [\[14\]](#) [\[21\]](#) [\[13\]](#) [\[12\]](#)

Concepts

- MCP servers expose data and tools (files, DBs, APIs) as a unified interface.^[12]
- MCP clients (assistants, agents) consume servers.^[12]
- Transport: stdio and HTTP with SSE using JSON-RPC 2.0.^{[14] [13]}

Why it Matters

- Avoids fragmented, per-platform tool integrations.
- Enables secure, standardized two-way communication between AI and systems across vendors.^{[13] [14] [12]}
- Adopted by Anthropic, OpenAI, and other providers.^{[21] [14] [13]}

Resources

- Anthropic MCP announcement – <https://www.anthropic.com/news/model-context-protocol>^[12]
- MCP spec and examples – <https://modelcontextprotocol.io>^{[14] [13]}
- OpenAI MCP adoption blog – <https://getathenai.com/blog/openai-model-context-protocol-announcement>^[21]

Projects

- MCP server for internal company data (Postgres, GitHub, Slack).
- MCP client integration inside an Agents SDK or LangGraph-based agent service.

AI Infrastructure

AI systems run on top of robust infra.^{[2] [1]}

Core Technologies

- Docker and containerization for services, with resource limits and health checks.^[2]
- Kubernetes for scaling AI workloads and managing deployments.^[1]
- Message queues: Kafka for high-throughput streams, RabbitMQ for reliable messaging.^[1]
- API gateways: NGINX, Traefik for routing and TLS.^[1]

Cloud Providers

- AWS (Lambda, ECS/EKS, S3, RDS, Bedrock).
- Azure (AKS, Cognitive Services).
- GCP (GKE, Vertex AI).^[1]

GPU & Inference

- GPU instances (NVIDIA A10/A100/H100) on cloud providers. ^[2]
- vLLM, TensorRT-LLM, Ray Serve, KServe for serving open-source models. ^{[11] [2]}

Databases for AI Systems

You already know PostgreSQL and Redis; AI systems add: ^[1]

- MongoDB for flexible document storage (conversation logs, configurations). ^[1]
- Elasticsearch for text search and analytics over logs and documents. ^[1]
- Neo4j for knowledge graphs and relationship reasoning. ^[1]
- Vector DBs (see Vector Databases section) for semantic retrieval. ^[1]

AI Security

Security is critical as AI acts on behalf of users. ^[4]

Risks

- Prompt injection and jailbreaks (malicious user content trying to override instructions). ^[4]
- PII leakage in logs and responses.
- Secrets and API key exposure via tools or logs.
- Rate limiting bypass and DoS on AI endpoints.
- RBAC and authorization issues for agents and tools. ^[4]
- Supply chain (model origin, dataset provenance, dependency vulnerabilities). ^[4]

Practices

- Input filtering and validation (schemas, policy filters). ^{[6] [7]}
- Output post-processing, redaction, and policy checks (e.g., safe completion filters). ^[4]
- Strict credential scoping and secret management using Vault/KMS. ^[1]
- Rate limiting and quotas per user/tenant in API gateway and service layer. ^[4]

Observability

Observability lets you understand cost, latency, and quality of AI features. ^{[8] [9]}

Langfuse

Langfuse is an open-source AI engineering platform for tracing, prompt management, and evaluation. ^[23] ^[9]

- Traces and spans with prompts, responses, tools, and metrics. ^[8]
- Token and cost tracking per feature and user. ^[8]
- Evaluation workflows and datasets plus dashboards. ^[9]

LangSmith

LangSmith provides visualization of agent graphs and prompts, integrated with LangChain and LangGraph. ^[10] ^[9]

- Agent graph views for multi-agent workflows.
- Prompt experiments and dataset evals.

Phoenix-style Local Observability

Phoenix and similar tools enable local-first LLM tracing, ideal during development. ^[22]

Checklist:

- [] Capture traces and metrics for all AI calls.
- [] Track latency, token usage, and cost per feature and user. ^[8]
- [] Integrate evaluation and dashboards into your CI/CD practice. ^[9]

Production AI Architecture

High-level components of a production AI platform. ^[4] ^[2]

```
flowchart LR
    GW[API Gateway] --> RT[LLM Router]
    RT -->|Hosted LLMs| H1[OpenAI/Anthropic]
    RT -->|Local LLMs| H2[vLLM/TensorRT-LLM]

    GW --> AG[Agent Orchestrator]
    AG --> Q[Queues (Kafka/RabbitMQ)]
    AG --> C[Cache (Redis)]

    AG --> RAG[RAG Service]
    RAG --> EMB[Embedding Workers]
    EMB --> VDB[Vector DB]
    RAG --> DB[Primary DB (Postgres/Mongo)]

    AG --> OBS[Observability (Langfuse/LangSmith)]
    AG --> SEC[Security & Guardrails]

    OBS --> MON[Dashboards & Alerts]
```

Key flows:

- API Gateway routes requests to LLM router or agent orchestrator and handles auth/rate limits.^[4]
- Agents coordinate tools, RAG, and long-running workflows via queues.^{[6] [10]}
- Embedding workers and vector DB handle retrieval and indexing.^{[4] [1]}
- Observability and security wrap the stack, providing visibility and protections.^{[8] [9] [4]}

Portfolio Projects (20 Projects)

Design 20 progressively harder projects. Each project should include:

- Description
- Difficulty (Beginner/Intermediate/Advanced)
- Skills
- Architecture
- Estimated Time
- Tech Stack
- GitHub Structure
- Deployment
- Resume Value

Example lineup (you can adapt):

1. **LLM-Powered FAQ Bot (Beginner)** – Single-LLM chat over static FAQs, hosted on FastAPI + Next.js.^[2]
2. **Doc RAG for Developer Docs (Intermediate)** – RAG over your own API docs, with evaluation and Langfuse tracing.^[9]
3. **E-commerce AI Search & Recommendations (Intermediate)** – Combine ElasticSearch, embeddings, and product ranking.^{[4] [1]}
4. **Multi-LLM Router Service (Intermediate)** – Route between OpenAI/Anthropic/Mistral based on cost/performance.^{[5] [2]}
5. **Agentic Customer Support Workflow (Advanced)** – Multi-agent pipeline for triage, answer generation, and escalation.^{[15] [10]}
6. **MCP-based Internal Knowledge Hub (Advanced)** – MCP server exposing company data plus AI assistant clients.^{[13] [12]}
7. **AI Code Reviewer for Node/TS Repos (Advanced)** – LLM agent reviewing PRs via GitHub API and MCP.^{[12] [7]}
8. **AI-Powered Analytics Narrator (Intermediate)** – LLM summarizing business dashboards for management.

9. **Multi-modal Product Assistant (Advanced)** – Vision + text transformer answering about images and descriptions. ^[2]
10. **AI Data Pipeline Monitor (Intermediate)** – LLM monitoring ETL logs and suggesting fixes.
11. **LangGraph-based Agent Platform (Advanced)** – Graph-based orchestration with pluggable agents and tools. ^[10]
12. **Fine-tuned Domain QA Model (Advanced)** – LoRA/QLoRA fine-tune on domain-specific data. ^[18]
13. **Self-hosted RAG Platform (Intermediate)** – RAG served via self-hosted vector DB and LLM. ^{[2] [1]}
14. **Streaming Chatbot with Observability (Intermediate)** – SSE/WebSockets + Langfuse tracing and dashboards. ^[9]
15. **OpenAI Agents SDK-based Workflow Engine (Advanced)** – Workflow definitions as code using Agents SDK. ^{[7] [6]}
16. **K8s-based AI Microservice Mesh (Advanced)** – Multi-service AI stack on Kubernetes with ingress and autoscaling. ^[1]
17. **AI Knowledge Graph Explorer (Neo4j) (Advanced)** – LLM answering questions over Neo4j graph. ^[1]
18. **AI Feature Flag and A/B Testing Platform (Advanced)** – Evaluate different prompts/models via controlled experiments. ^[9]
19. **Security-hardened RAG System (Advanced)** – Focus on OWASP LLM Top 10 mitigations. ^[4]
20. **Company-wide AI Copilot Platform (Advanced)** – Multi-feature copilot integrated into internal tools. ^{[4] [2]}

Open Source Contributions

Target high-impact repositories that align with RAG, agents, and observability. ^{[23] [10] [11]}

- LangChain – <https://github.com/langchain-ai/langchain> ^[10]
- LangGraph – <https://github.com/langchain-ai/langgraph> ^[10]
- LlamaIndex – https://github.com/run-llama/llama_index ^[1]
- Open WebUI – <https://github.com/open-webui/open-webui> ^[2]
- Ollama – <https://github.com/ollama/ollama> ^[2]
- vLLM – <https://github.com/vllm-project/vllm> ^[2]
- Qdrant – <https://github.com/qdrant/qdrant> ^[1]
- Haystack – <https://github.com/deepset-ai/haystack> ^[1]
- FastAPI – <https://github.com/tiangolo/fastapi> ^[1]
- Transformers – <https://github.com/huggingface/transformers> ^[18]
- Sentence Transformers – <https://github.com/UKPLab/sentence-transformers> ^[18]

Guidance:

- Start with documentation, examples, and small bug fixes. ^[23] ^[10]
- Move to features and integration examples relevant to your portfolio. ^[11]

Interview Preparation

Areas

- System design (AI features, RAG, agents, observability).
- Coding (Python, algorithms, data structures).
- LLMs and RAG design, failure modes, cost optimization. ^[2] ^[1]
- Agents and orchestration (LangGraph, Agents SDK). ^[6] ^[10]
- SQL and data modeling (Postgres, indexing).
- Prompt engineering and evaluation strategies. ^[6] ^[9]
- Architecture and infra (Docker, K8s, queues, caching). ^[2] ^[1]
- Behavioral (impact, ownership, communication). ^[3]

100 Interview Questions (Outline)

Sample categories:

- Explain transformer attention and context windows. ^[11] ^[2]
- Design a RAG system for an internal knowledge base. ^[4] ^[1]
- Compare pgvector vs Qdrant vs Pinecone for a multi-tenant app. ^[1]
- How to prevent prompt injection in a RAG chatbot? ^[4]
- Design an agent that books flights and hotels via APIs (tools, state, approvals). ^[7] ^[6]
- How do you measure and improve LLM output quality in production? ^[9]
- When would you fine-tune vs rely on prompting and RAG? ^[18] ^[2]
- Trade-offs between hosted LLMs and self-hosted vLLM/TensorRT. ^[11] ^[2]

Career Roadmap

Levels and typical expectations. ^[2] ^[3] ^[1]

- **Junior AI Engineer** – Strong Python and basic LLM/RAG, works on features under guidance.
- **Mid AI Engineer** – Owns features end-to-end, understands RAG, vector DBs, and basic agents.
- **Senior AI Engineer** – Designs systems, mentors others, handles incidents, influences architecture.
- **Staff AI Engineer** – Cross-team impact, drives AI platform initiatives, deep expertise in multiple subsystems.

- **Principal AI Engineer** – Strategy-level decisions, long-term platform evolution and multi-cloud architectures.
- **AI Architect** – Focused on overall AI system architecture, standards, and integration across org. ^[8] ^[4]

Certifications

Ranked broadly by practical ROI for AI Engineering. ^[11] ^[3]

- AWS AI/ML specialty – strong baseline for cloud + ML. ^[3]
- Google Cloud AI (Vertex AI) – good if you target GCP-heavy orgs. ^[3]
- Azure AI Engineer – relevant in Microsoft-centric environments. ^[3]
- NVIDIA certifications focused on CUDA/TensorRT and GPUs. ^[11]
- Hugging Face courses and badges around transformers/RAG. ^[18]
- DeepLearning.AI specializations (LLMs, RAG, ML, DL). ^[3]
- Databricks ML/AI certifications for data-heavy orgs. ^[3]

Use these as supplements; portfolio and experience are higher ROI. ^[4] ^[2]

Books

Rank best books by difficulty and importance for AI Engineering. ^[3] ^[1]

- “Python Crash Course” (Eric Matthes) – Beginner, high importance as Python foundation. ^[1]
- “Fluent Python” (Luciano Ramalho) – Intermediate/Advanced, high importance for idiomatic Python. ^[1]
- “Mathematics for Machine Learning” – Intermediate, medium importance for math. ^[3]
- “Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow” – Intermediate, high importance for ML/DL foundations. ^[3]
- “Deep Learning” (Goodfellow, Bengio, Courville) – Advanced, high importance for deep learning theory. ^[3]
- “Designing Machine Learning Systems” (Chip Huyen) – Intermediate, high importance for ML system design. ^[3]
- **Modern 2025/2026 books on LLM apps, RAG, and agents** – look for recent titles focused on production LLM systems. ^[2] ^[1]

Research Papers

Foundational and modern papers to understand LLMs, RAG, and agents. ^[18] ^[11] ^[2]

- **Attention Is All You Need** – Introduced transformer architecture, basis for modern LLMs, key to understanding attention. ^[11]
- **BERT** – Bidirectional encoder for language understanding; inspired many downstream tasks and models. ^[18]
- **GPT series** – Autoregressive transformer models demonstrating scaling laws and generative capabilities. ^[2]
- **RoBERTa** – Robustly optimized BERT with improved pretraining, highlights importance of training regimes. ^[18]
- **LoRA** – Low-Rank Adaptation enabling parameter-efficient fine-tuning, crucial for customizing large models. ^[18]
- **QLoRA** – Extends LoRA by using quantization to fine-tune models on limited hardware. ^[18]
- **Llama, Mistral, Mixtral papers** – Modern open-source LLM families with strong performance and efficient architectures. ^[5] ^[2]
- **RAG-related papers** – Research on retrieval-augmented generation demonstrating benefits of grounding and retrieval strategies. ^[4]
- **Agent-related papers** – Work on multi-agent setups, orchestration frameworks, and task decomposition. ^[25] ^[15]

Each paper matters because it either introduces key architectures (transformers, BERT, GPT), improved training/fine-tuning methods (LoRA, QLoRA), or open models powering modern AI stacks (Llama, Mistral, Mixtral). ^[18] ^[11] ^[2]

Best Learning Resources

Curate high-quality resources with direct links. ^[2] ^[3] ^[1]

Official Documentation

- OpenAI API Docs – <https://platform.openai.com/docs> ^[7] ^[6]
- Anthropic Docs – <https://docs.anthropic.com/> ^[12]
- Hugging Face – <https://huggingface.co/docs> ^[18]
- LangChain & LangGraph Docs – <https://docs.langchain.com/> ^[10]
- Langfuse Docs – <https://langfuse.com/docs> ^[9]

GitHub Repositories

- LangGraph – <https://github.com/langchain-ai/langgraph>^[10]
- vLLM – <https://github.com/vllm-project/vllm>^[2]
- Awesome LangGraph – <https://github.com/von-development/awesome-LangGraph>^[11]
- Langfuse – <https://github.com/langfuse/langfuse>^[23]

YouTube Playlists

- AI Engineer Roadmap videos (2026 roadmaps and series).^{[26] [27] [5] [2]}

Free & Paid Courses

- [DeepLearning.AI](#) LLM/RAG courses.^[3]
- DataCamp AI Engineer tracks.^{[27] [3]}
- LangChain Academy courses for LangGraph and agents.^[10]

Blogs, Newsletters, Podcasts

- AI engineering blogs focusing on LLM apps, RAG, and agents.^{[28] [8]}
- Newsletters summarizing new LLM releases and papers.
- Podcasts on applied AI engineering and case studies.

Communities

- Discord servers for LangChain, LangGraph, Langfuse.^{[23] [9]}
- Reddit communities [r/MachineLearning](#), [r/LocalLLaMA](#).^[3]

ArXiv & Papers with Code

- [arxiv.org](#) for latest papers.^{[18] [11]}
- [paperswithcode.com](#) for code implementations.^[18]

Hugging Face & Kaggle

- Hugging Face for models, datasets, and spaces.^[18]
- Kaggle for datasets and competitions to practice ML.^[3]

Daily Study Routine

Assuming 20 hours/week.^{[2] [1]}

- Morning (1–2h) – Reading docs, papers, and blogs; taking notes.
- Afternoon (2–3h) – Hands-on coding and experiments aligned to weekly plan.

- **Evening (1-2h)** – Project work, refactoring, documentation.
- **Weekend (4-6h)** – Deep work on portfolio projects, open source, and interview prep.

Include:

- Revision of key concepts (math, ML, transformers).
- Interview practice (system design, coding).
- Reading papers and recent blog posts.
- Open source contributions (issues, PRs). ^[23] ^[10]

Progress Tracking

Checklists

Create checklists per area:

- Python & FastAPI
- ML & DL
- Transformers & LLMs
- RAG & Vector DBs
- Agents & MCP
- Infra & K8s
- Observability & Security

Milestone Tracker

Track milestones per month:

- Month 1 – Python & math foundations.
- Month 2 – ML/DL basics.
- Month 3 – LLMs and simple RAG.
- Month 4 – Advanced RAG and vector DBs.
- Month 5 – Agents, MCP, infra.
- Month 6 – Portfolio, interviews, open source. ^[2] ^[1]

Completion Tracker

Track completion of:

- Weekly plans (24 weeks).
- Portfolio projects (20).
- Open source PRs (minimum 3-5). ^[23] ^[10]

Skill Matrix

Create a matrix of skills vs proficiency (Beginner, Intermediate, Advanced) for:

- Python, FastAPI, ML, DL, transformers, LLM APIs, RAG, vector DBs, agents, infra, security, observability.

Knowledge Graph

Map concepts as nodes and edges (e.g., LLM → Transformers → Attention → Embeddings → RAG → Agents → MCP → Observability).^[8] ^[4]

Final Checklist

Complete this checklist to graduate as a Production AI Engineer in 2026.^[2] ^[1] ^[3]

- ☐ Python (typed, tested, production-ready)
- ☐ FastAPI & Pydantic
- ☐ PyTorch & TensorFlow basics
- ☐ Transformers (attention, tokenization, embeddings)
- ☐ LLM APIs (OpenAI, Anthropic, others)
- ☐ Prompt Engineering (system, tools, templates)
- ☐ Structured Output & Tool Calling
- ☐ RAG (embeddings, chunking, hybrid search, reranking)
- ☐ Vector Databases (pgvector, Qdrant, Pinecone, etc.)
- ☐ AI Agents (LangGraph, Agents SDK, PydanticAI, CrewAI, SmolAgents)
- ☐ MCP (servers, clients, tools, security)
- ☐ Docker & Kubernetes for AI workloads
- ☐ AWS/Azure/GCP for AI deployment
- ☐ Inference Servers (vLLM, TensorRT-LLM, Ray Serve, KServe)
- ☐ Databases (Postgres, Redis, MongoDB, Elasticsearch, Neo4j)
- ☐ AI Security (prompt injection, jailbreaks, PII, secrets, RBAC)
- ☐ Observability (Langfuse, LangSmith, Phoenix-style tools)
- ☐ Production AI Architecture (gateway, router, workers, queues, DBs, observability)
- ☐ Portfolio (3–5 strong, deployed projects)
- ☐ Open Source Contributions (PRs to key repos)
- ☐ Interview Ready (system design, coding, AI rounds)

Once you can **demonstrate** these with working code, deployed systems, and clear documentation, you are ready to operate as a Production AI Engineer in 2026.^[4] ^[2] ^[1]



1. <https://www.scaler.com/blog/ai-engineer-roadmap-master-genai-llms-deep-learning/>
2. <https://www.youtube.com/watch?v=4UAlNTLhG9I>
3. <https://www.turingcollege.com/blog/ai-engineer-roadmap-how-to-become-an-ai-engineer>
4. https://www.linkedin.com/posts/brijpandeyji_how-to-become-an-ai-engineer-in-2026-without-activity-7420712399294398464-dXLn
5. <https://www.youtube.com/watch?v=7Gxu-VCPJ0A>
6. <https://developers.openai.com/api/docs/guides/agents>
7. <https://platform.openai.com/docs/guides/agents-sdk>
8. <https://langfuse.com/docs/observability/overview>
9. <https://langfuse.com/docs>
10. <https://github.com/langchain-ai/langgraph>
11. <https://github.com/von-development/awesome-LangGraph>
12. <https://www.anthropic.com/news/model-context-protocol>
13. https://cs.wikipedia.org/wiki/Model_Context_Protocol
14. https://vi.wikipedia.org/wiki/Model_Context_Protocol
15. <https://medium.com/@mpuig/which-multi-agent-framework-should-run-your-enterprise-ai-abdc8e09ad89>
16. <https://github.com/pydantic/pydantic-ai>
17. <https://github.com/pydantic>
18. <https://github.com/topics/langgraph>
19. <https://ianas.fr/en/blog/2026/06/02/crewai-langchain-langgraph-comparatif-pragmatique/>
20. <https://github.com/langchain-ai/langgraph-101>
21. <https://getathenic.com/blog/openai-model-context-protocol-announcement>
22. <https://michaeljohnpena.com/blog/2024-10-18-phoenix-observability/>
23. <https://github.com/langfuse/langfuse>
24. <https://github.com/topics/pydanticai>
25. <https://langfuse.com/blog/2025-03-19-ai-agent-comparison>
26. <https://www.youtube.com/watch?v=TOHNb42t0iE>
27. <https://www.youtube.com/watch?v=1Eq4El-XpIg>
28. <https://www.codercops.com/blog/llm-observability-langfuse-helicone-2026/>
29. <https://www.braincuber.com/tutorial/ai-engineer-roadmap-2026-complete-guide>
30. <https://github.com/snr177/langgraph>
31. <https://github.com/langchain-ai/langgraph-example>
32. <https://github.com/emarco177/langgraph-course>
33. <https://github.com/iamvaibhavmehra/LangGraph-Course-freeCodeCamp>
34. <https://github.com/langchain-ai/new-langgraph-project/blob/main/langgraph.json>

35. <https://github.com/LangGraph-GUI/LangGraph-learn>
36. [https://huggingface.co/api/resolve-cache/spaces/arterm-sedov/cmw-copilot/ee398ad8b178829dffb3d035ccbf8920571210b/docs%2F20250922_LANGSMITH_SETUP_COMPLETE.md?download=true&etag="07758586cd9544f25deb6aecf8b33728ef63e4b0"](https://huggingface.co/api/resolve-cache/spaces/arterm-sedov/cmw-copilot/ee398ad8b178829dffb3d035ccbf8920571210b/docs%2F20250922_LANGSMITH_SETUP_COMPLETE.md?download=true&etag=)
37. <https://github.com/orgs/langfuse/discussions/11391>
38. <https://voce.com/@patriciajames/observability-langfuse-2026-developer-guide-hxujqk>
39. <https://www.shareuhack.com/en/posts/llm-agent-observability-langfuse-guide-2026>
40. <https://langfuse.com>
41. <https://www.mindmapdigital.ai/blog/langfuse-observability>
42. <https://github.com/orgs/langfuse/discussions/12518>
43. <https://qaskills.sh/blog/langfuse-llm-observability-guide-2026>
44. <https://www.marktechpost.com/2026/05/24/build-a-complete-langfuse-observability-and-evaluation-pipeline-for-tracing-prompt-management-scoring-and-experiments/>
45. <https://effloow.com/articles/langfuse-llm-observability-self-host-guide-2026>